

Name: Kanika Saini

SAP ID: 590034098

Batch: B16

Program: Btech CSE

Date: 18-08-2026

Experiment -1

Linux Programming Environment & Basic Commands

A) Setting up of Linux Programming Environment

We can do C programming online or offline according to the needs. It is required for us to keep working on a Linux Environment.

Online: Cocalc.ai –

1. Open CoCalc.ai and sign in.
2. Create/open a project and launch a Linux terminal.
3. Create a C source file (for example, hello.c).
4. Write and save the C program.
5. Compile the program using gcc.
6. Execute the program and observe the output.

CoCalc.ai project/terminal showing the Linux environment and your C program.

```
~$ vi hello.c
~$ cc hello.c -o hello
~$ ./hello
Hello World
~$ █
```

Offline:

Methods: Dual boot, Single boot, Bootable Pendrive, VM

Dual Boot – Two operating systems are installed on the same computer and the user selects one while starting the computer.

Single Boot – One operating system is installed and used as the main operating system.

Bootable Pendrive – A USB drive containing a bootable operating system is used to start the computer.

Virtual Machine (VM) – A virtual computer is created inside an existing operating system using virtualization software.

1. Virtual Machine Manager (Hypervisor)

Type-1 (Baremetal)

It is installed directly on the computer hardware. It does not require another operating system between the hardware and hypervisor.

Working:

Hardware → Hypervisor → Virtual Machines → Operating Systems

Examples: VMware ESXi, Microsoft Hyper-V Server, Xen.

Main point: It provides better performance and efficiency and is commonly used on servers and data centers.

Type-2 (Hosted)

It is installed on an existing operating system, just like normal software.

For example, if we have Windows, we can install Oracle VirtualBox on Windows and then install Kali Linux inside VirtualBox.

Working:

Hardware → Host Operating System → Hypervisor → Virtual Machine → Guest OS

Examples: Oracle VirtualBox, VMware Workstation, VMware Fusion.

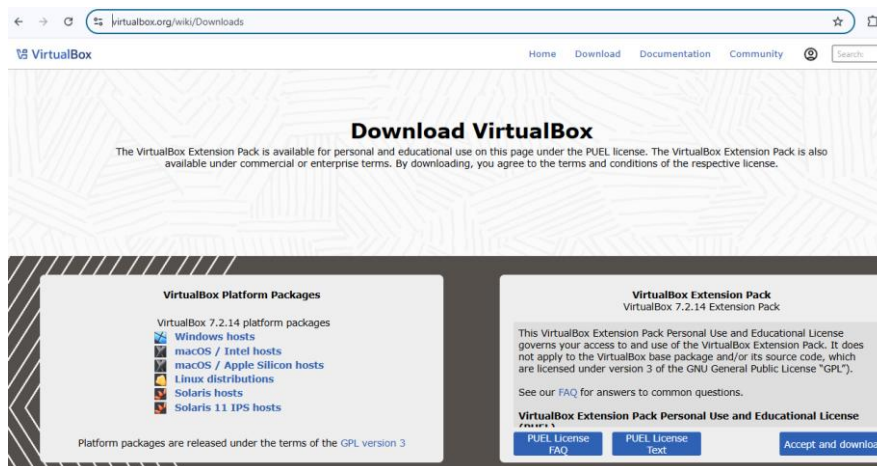
Main point: It is easy to install and use, so it is commonly used for learning, testing, and programming.

In Our C Programming Project.

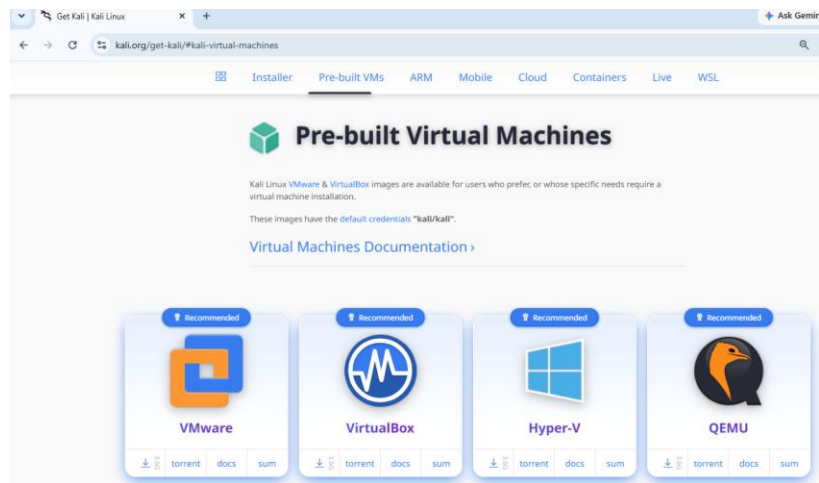
We are using Oracle VirtualBox, which is a Type 2 (Hosted) Hypervisor. We will install Kali Linux as a virtual machine inside VirtualBox and use it for C programming.

2. Installation of VMM & VM

Step-1: Download Oracle Virtual Box for “Windows hosts”

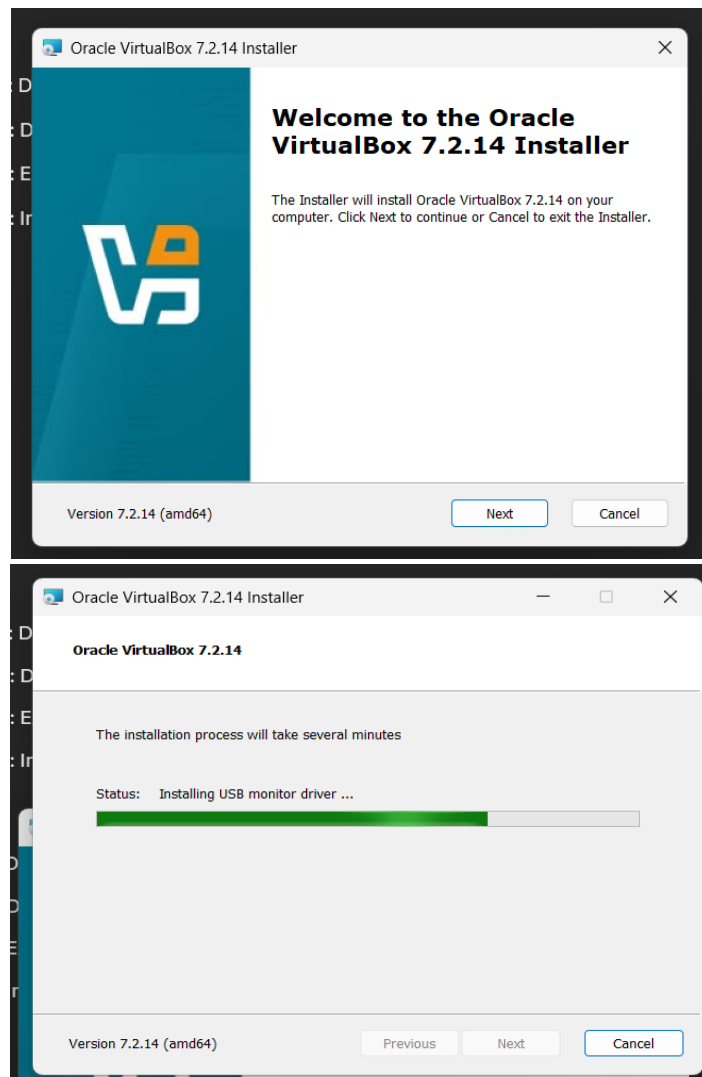


Step-2: Download Pre-built Kali Linux VM for Virtualbox

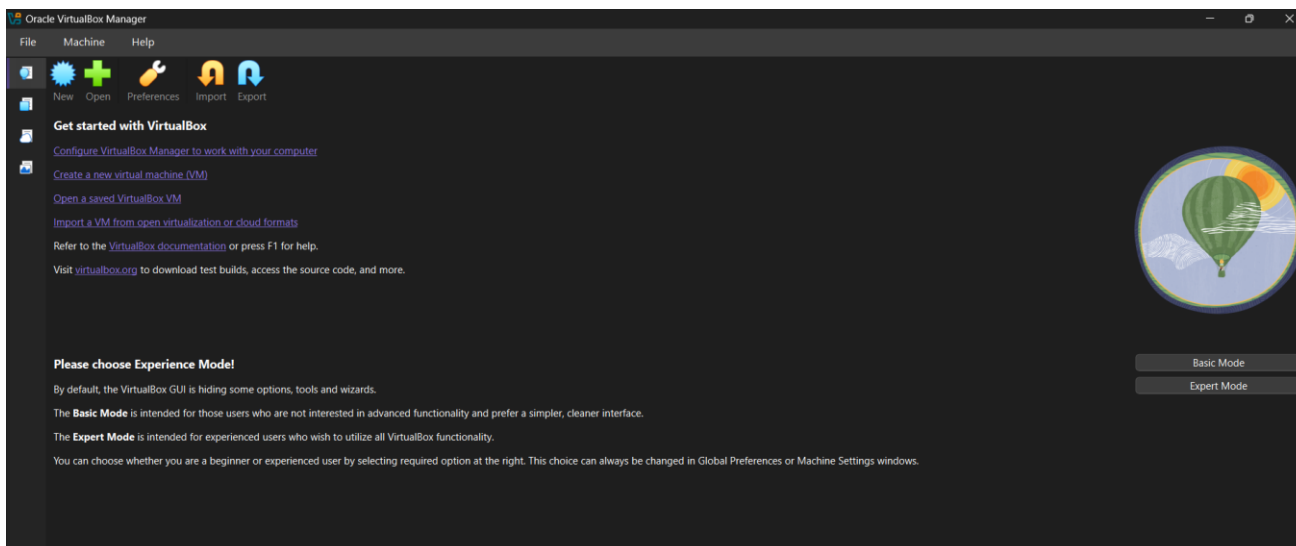


Step-3: Extracted the Kali Linux downloaded file

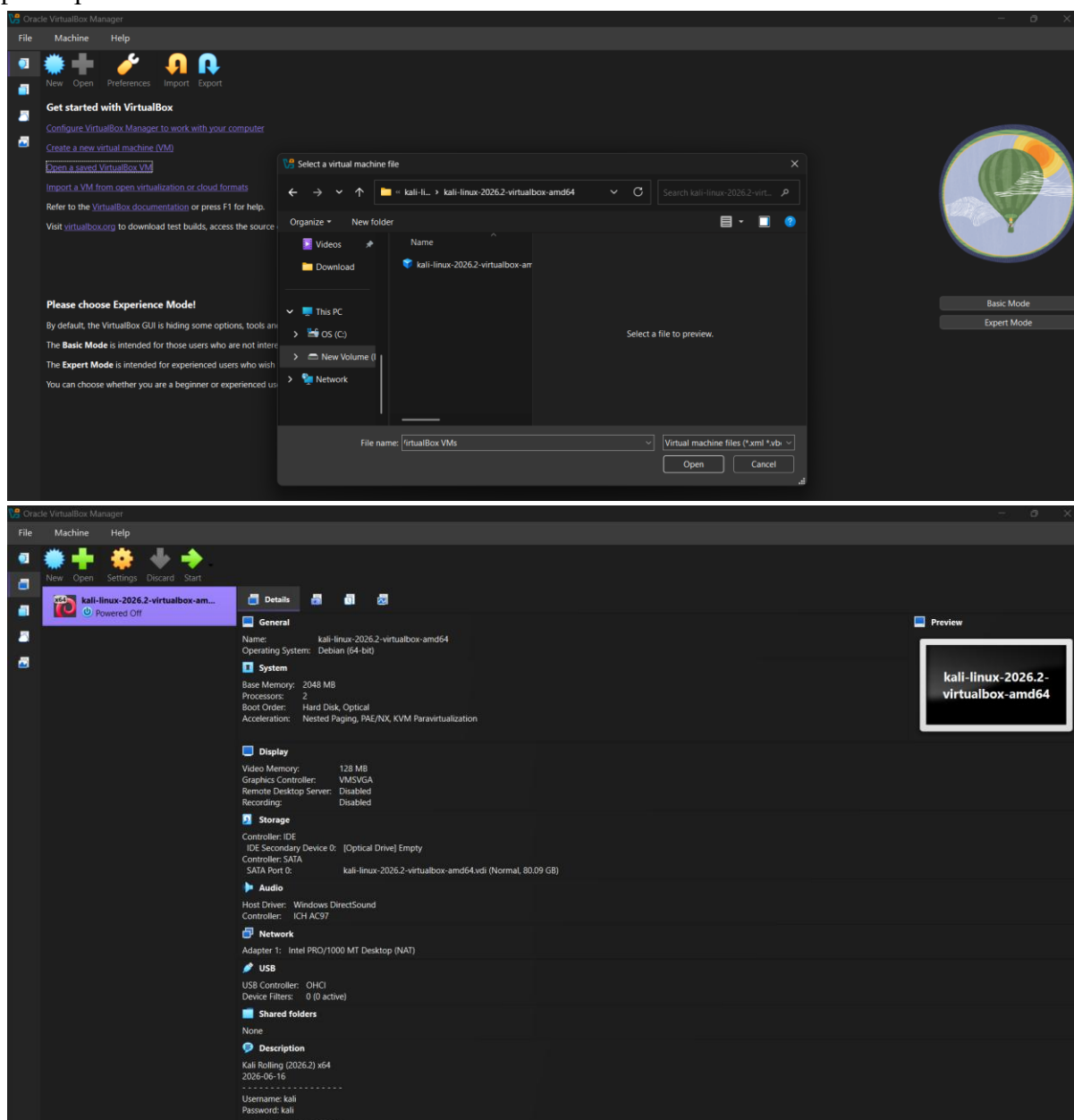
Step-4: Install Virtualbox



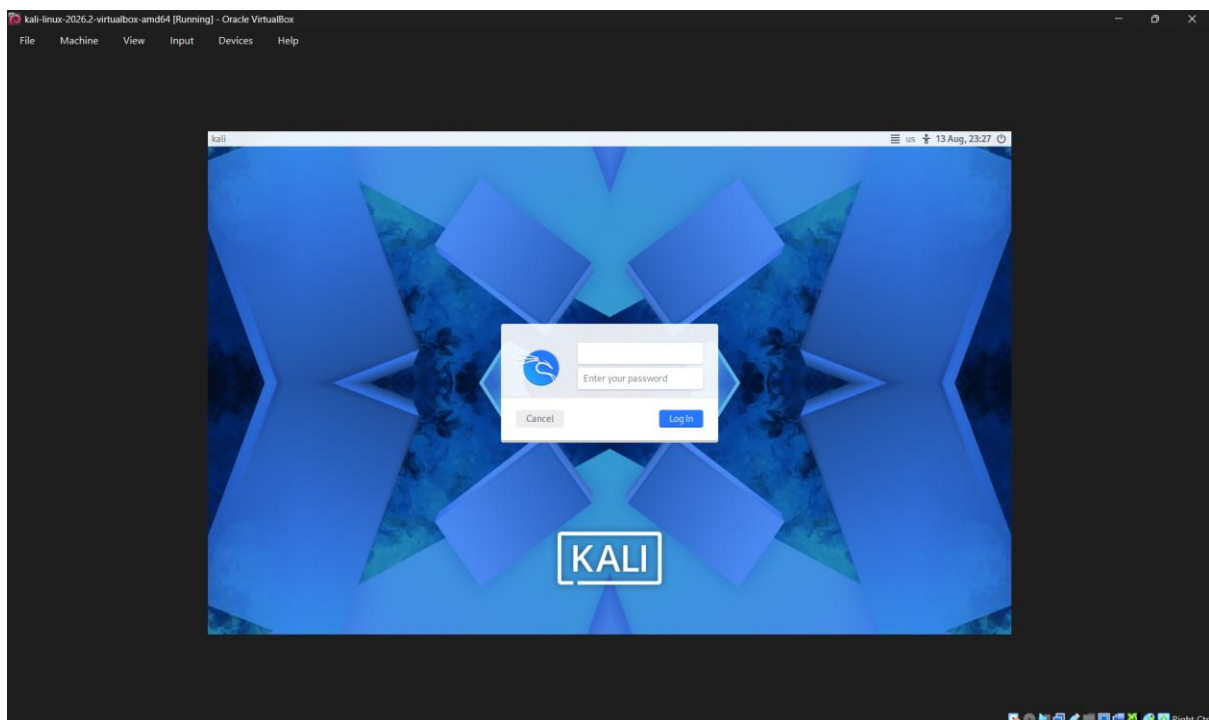
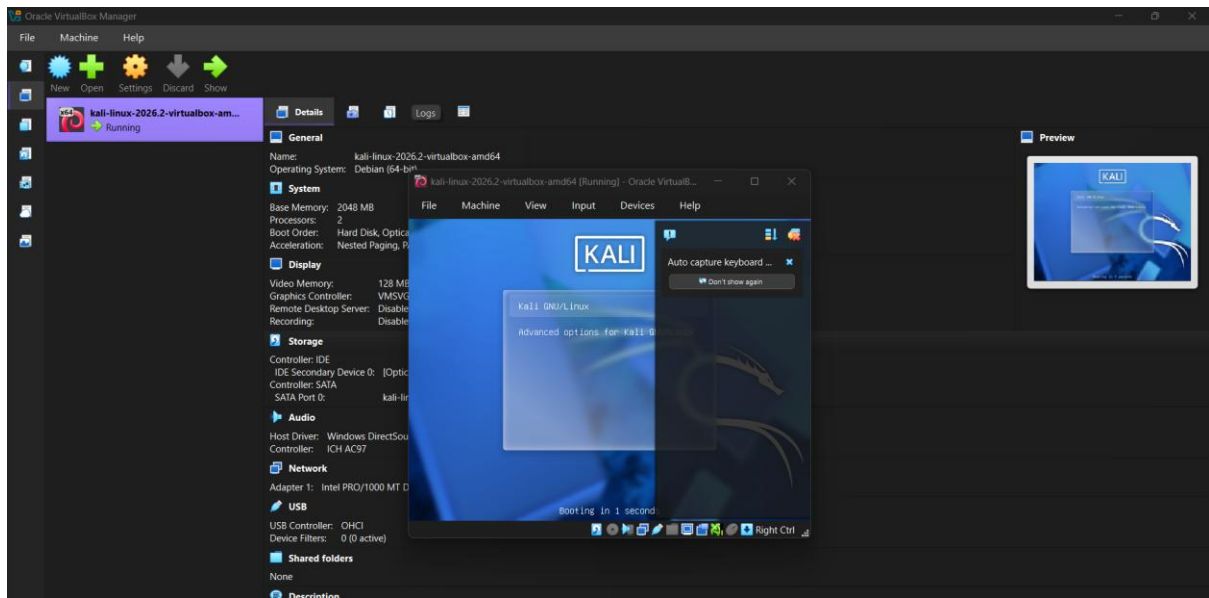
Step-5: Open Virtual box



Step-6: Open the “saved VM”



Step-7: Start the VM



Thus, the installation of VMM (Hypervisor Type-2 Oracle VirtualBox) & VM (Kali Linux) is completed.

Step-8: Open the terminal and create the following simple program, compile and execute to understand the working of the environment.

```
#include <stdio.h>

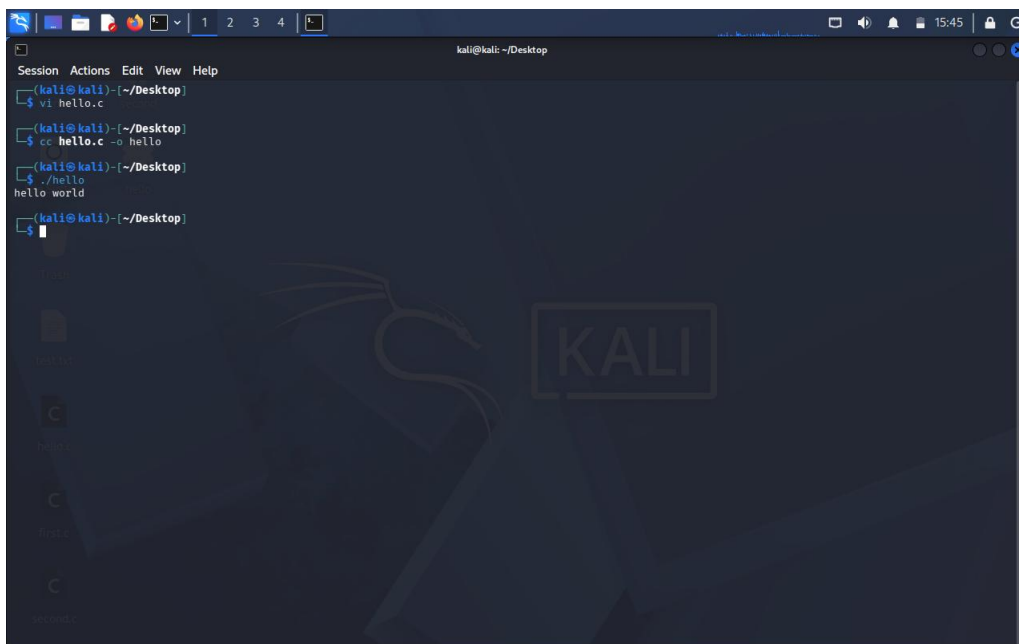
int main()
{
    printf("Hello World");
    return 0;
}
```

Compile command: gcc hello.c -o hello

Execution command: ./hello

Expected output: Hello World

Terminal showing the C program, compilation/execution and “Hello World” output.



```
kali@kali: ~/Desktop
$ vi hello.c
$ cc hello.c -o hello
$ ./hello
hello world
$
```

B) Working with Linux Commands

Execute the following commands in your environment, take output snippets, and provide a brief explanation.

Terminal basics

pwd – Shows the present working directory.

mkdir – Creates a new directory.

rmdir – Removes an empty directory.

dir – Lists files and directories.

ls – Lists files and directories.

cd – Changes the current directory.

cd .. – Moves to the parent directory.

clear – Clears the terminal screen.

history – Shows previously executed commands.

man – Shows the manual/help page of a command.

whoami – Shows the current username.

The terminal output screenshot for the terminal basics commands.

```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ pwd
/home/kali/Desktop
(kali@kali)-[~/Desktop]
$ ls
first.c  second  second.c
(kali@kali)-[~/Desktop]
$ mkdir test
(kali@kali)-[~/Desktop]
$ ls
first.c  second  second.c  test
(kali@kali)-[~/Desktop]
$ cd test
(kali@kali)-[~/Desktop/test]
$ pwd
/home/kali/Desktop/test
(kali@kali)-[~/Desktop/test]
$ cd ..
(kali@kali)-[~/Desktop]
$ pwd
/home/kali/Desktop
(kali@kali)-[~/Desktop]
$ rmdir test
(kali@kali)-[~/Desktop]
$ ls
first.c  second  second.c
(kali@kali)-[~/Desktop]
$ history
1  pwd
2  mkdir
3  mkdir demo1 demo2
4  dir
5  rmdir demo2
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help
8  dir
9  vi first.c
10 demo1 first.c
11 cd /home/kali/demo1
12 pwd
13 mkdir demo1
14 dir
15 clear
16 pwd
17 mkdir demo1
18 clear
19 mkdir demo2
20 dir
21 vi first.c
22 more first.c
23 cc first.c
24 vi first.c
25 ./a.out
26 dir
27 ./a.out
28 vi second.c
29 cc second.c -o second
30 ./second
31 pwd
32 ls
33 mkdir test
34 ls
35 cd test
36 pwd
37 cd ..
38 pwd
39 rmdir test
40 ls
(kali@kali)-[~/Desktop]
$ man pwd
(kali@kali)-[~/Desktop]
$ pwd
/home/kali/Desktop
(kali@kali)-[~/Desktop]
$
```

Files

vi – Opens the vi text editor.

file – Shows information about a file type.

rm – Removes files/directories.

cp – Copies files/directories.

mv – Moves or renames files/directories.

cat – Displays file contents.

more – Displays contents page by page.

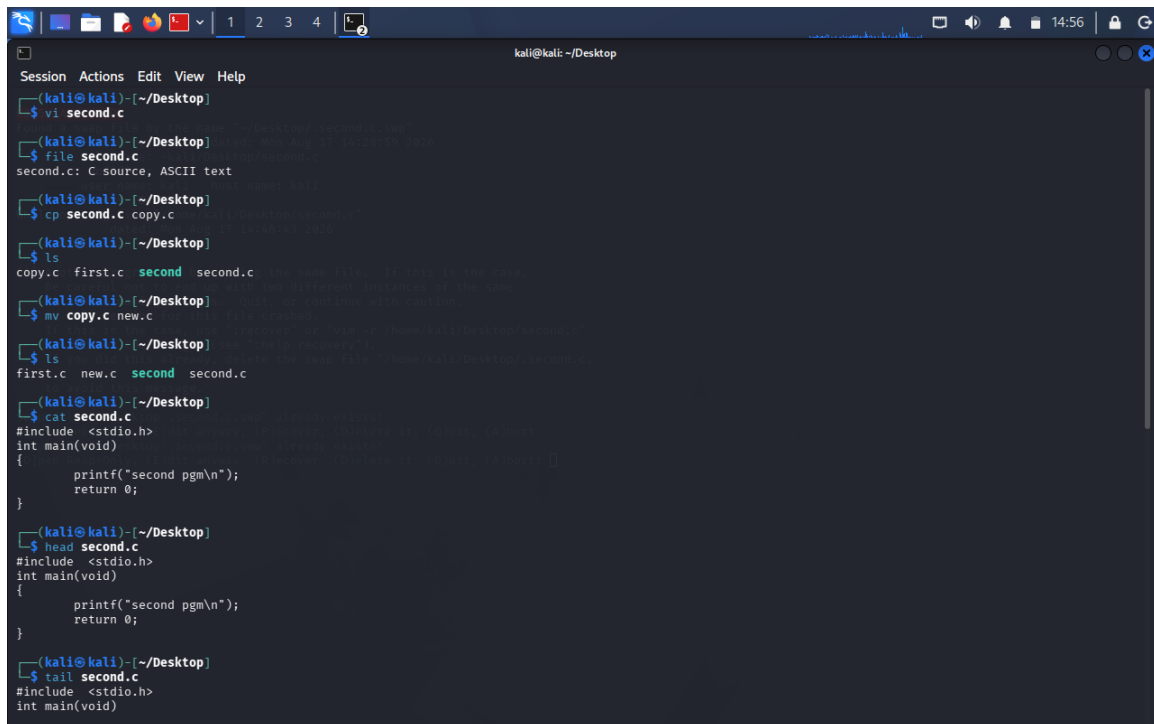
less – Views file contents with navigation.

head – Displays the beginning of a file.

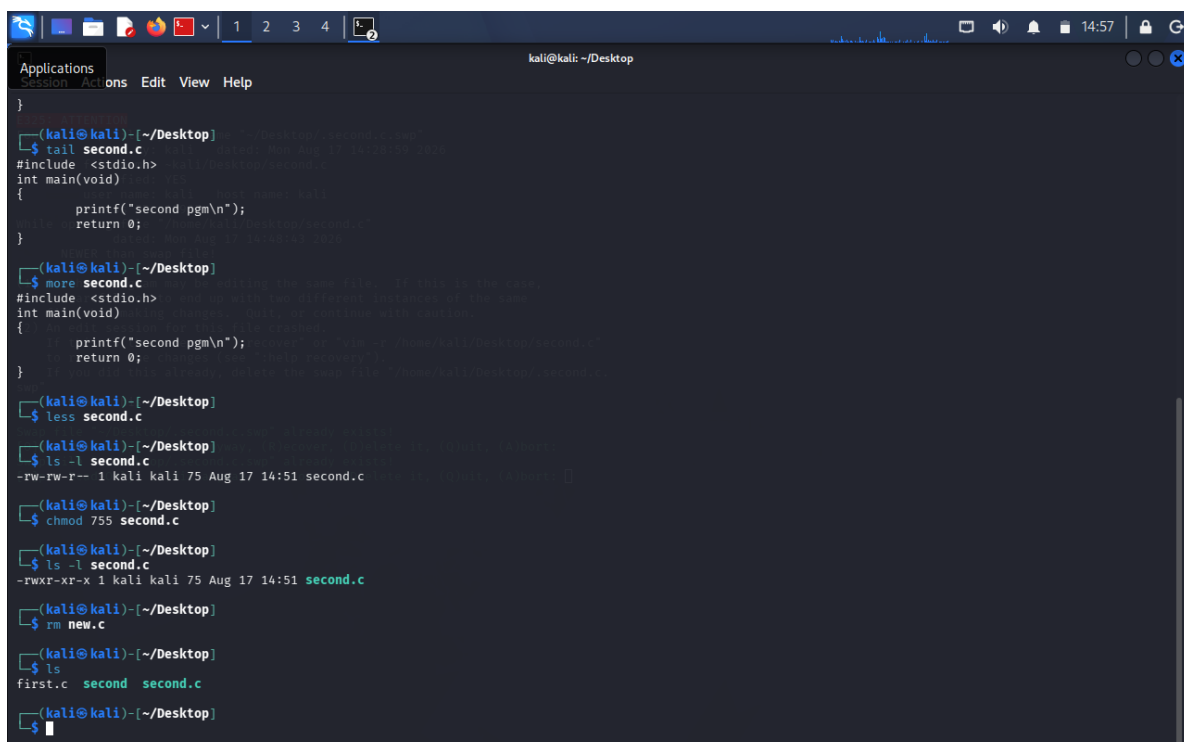
tail – Displays the end of a file.

chmod – Changes file permissions.

The terminal output screenshot for the files commands.



```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)~[~/Desktop]
$ vi second.c
(kali@kali)~[~/Desktop]
$ file second.c
second.c: C source, ASCII text
(kali@kali)~[~/Desktop]
$ cp second.c copy.c
(kali@kali)~[~/Desktop]
$ ls
copy.c first.c second second.c
(kali@kali)~[~/Desktop]
$ mv copy.c new.c
(kali@kali)~[~/Desktop]
$ ls
first.c new.c second second.c
(kali@kali)~[~/Desktop]
$ cat second.c
#include <stdio.h>
int main(void)
{
    printf("second pgm\n");
    return 0;
}
(kali@kali)~[~/Desktop]
$ head second.c
#include <stdio.h>
int main(void)
{
    printf("second pgm\n");
    return 0;
}
(kali@kali)~[~/Desktop]
$ tail second.c
#include <stdio.h>
int main(void)
```



```
kali@kali: ~/Desktop
Applications
Session Actions Edit View Help
(kali@kali)~[~/Desktop]
$ tail second.c
#include <stdio.h>
int main(void)
{
    printf("second pgm\n");
    return 0;
}
(kali@kali)~[~/Desktop]
$ more second.c
#include <stdio.h>
int main(void)
{
    printf("second pgm\n");
    return 0;
}
(kali@kali)~[~/Desktop]
$ less second.c
(kali@kali)~[~/Desktop]
$ ls -l second.c
-rw-rw-r-- 1 kali kali 75 Aug 17 14:51 second.c
(kali@kali)~[~/Desktop]
$ chmod 755 second.c
(kali@kali)~[~/Desktop]
$ ls -l second.c
-rwxr-xr-x 1 kali kali 75 Aug 17 14:51 second.c
(kali@kali)~[~/Desktop]
$ rm new.c
(kali@kali)~[~/Desktop]
$ ls
first.c second second.c
(kali@kali)~[~/Desktop]
$
```

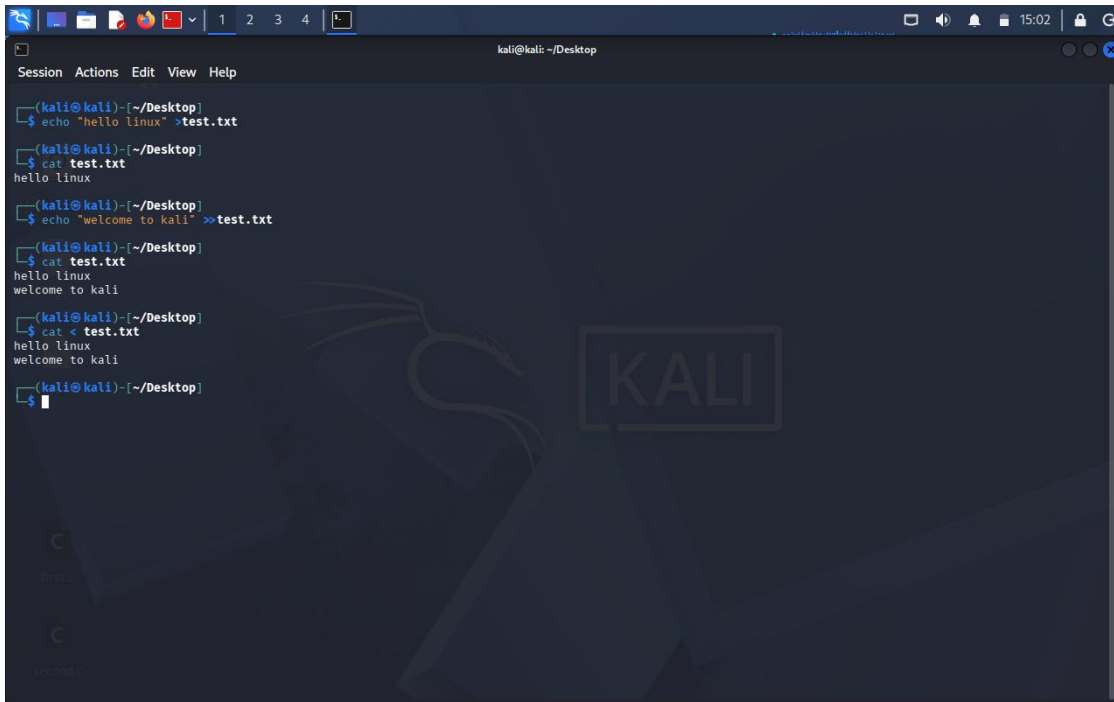

Redirection

> – Redirects output to a file, replacing its contents.

>> – Redirects output and appends it to a file.

< – Takes input from a file.

The terminal output screenshot for the redirection commands.

A terminal window titled 'kali@kali: ~/Desktop' showing a series of commands and their outputs. The commands demonstrate file redirection: creating a file with 'echo', appending to it with '>>', and reading from it with '<'. The background features a faint Kali Linux logo.

```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ echo "hello linux" >test.txt
(kali@kali)-[~/Desktop]
$ cat test.txt
hello linux
(kali@kali)-[~/Desktop]
$ echo "welcome to kali" >>test.txt
(kali@kali)-[~/Desktop]
$ cat test.txt
hello linux
welcome to kali
(kali@kali)-[~/Desktop]
$ cat < test.txt
hello linux
welcome to kali
(kali@kali)-[~/Desktop]
$
```

Result

The Linux programming environment was set up and the basic Linux commands were executed successfully